REAL-TIME COMMERCE STREAMING PIPELINE

Project Explanation & Data Engineering Portfolio Guide

GitHub Repository

Real-Time Commerce Streaming Pipeline — Project Explanation

1. Overview

This Data Engineering project simulates an e-commerce system where customer activities—product views, cart additions, checkouts, and purchases—are emitted as events and processed continuously. The pipeline demonstrates event ingestion, validation, idempotent storage, dead-letter handling, and SQL analytics.

2. Architecture

1. **Python producer:** generates synthetic customer activity events.

2. **Redpanda:** Kafka-compatible streaming broker; stores events in topics.

3. **Python consumer:** reads and validates messages.

4. **PostgreSQL:** stores valid event records.

5. **Dead-letter topic (DLQ):** holds invalid events and error context for investigation.

6. **SQL analytics:** queries purchase activity, revenue, product categories, and active users.

Flow: Producer → Redpanda → Consumer → PostgreSQL for valid events, or DLQ topic for invalid events.

3. Technologies

• **Python:** event generation, validation, consumption, and error handling.

• **Redpanda:** decouples producers from consumers with Kafka-compatible streaming.

• **PostgreSQL:** relational storage for validated events and analytics.

• **Docker Compose:** defines local services.

• **pytest:** automated tests for validation and database behavior.

4. Example event

```
{
  "event_id": "evt-1001",
  "event_type": "purchase",
  "event_time": "2026-10-23T10:00:00+00:00",
  "user_id": "user-25",
  "session_id": "session-501",
  "product_id": "SKU-101",
  "category": "electronics",
  "quantity": 2,
  "unit_price": 500
}
```

The consumer validates the payload, calculates revenue as quantity × unit price (1,000 in this example), and inserts the event into PostgreSQL.

5. Invalid data and reliability

A malformed timestamp is rejected and routed to `commerce.events.dlq` with error context. PostgreSQL uses `event_id` as a primary key and `ON CONFLICT DO NOTHING` to prevent duplicate rows. Consumer offsets are committed after message handling.

Important: Idempotent inserts prevent duplicate rows, but do not alone guarantee end-to-end exactly-once processing. Stronger guarantees require coordinating database transactions, offsets, and failure recovery.

6. Analytics

The `sql/analytics.sql` file includes queries for purchase revenue by hour, unique sessions by event type, revenue by category, and daily active users.

7. Run on Windows

Prerequisites: Git, Docker Desktop, and Docker Compose. In PowerShell:

```
git clone https://github.com/shubham05jaiswal-coder/realtime-commerce-streaming-pipeline.git
cd realtime-commerce-streaming-pipeline
Copy-Item .env.example .env
docker compose up --build -d
docker compose ps
docker compose logs -f consumer
```

Publish 100 generated events in another terminal:

```
docker compose run --rm consumer python -m src.producer --count 100
```

Run tests and the offline demo in a local Python environment:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pytest -q
python -m src.demo
```

The full Docker integration has not been independently verified in a running environment. Inspect service health and logs before assuming all components work together.

8. Data engineering concepts demonstrated

• Streaming ingestion and event-driven architecture

• Schema validation and data quality

• Idempotent writes and duplicate prevention

• Dead-letter handling

• Consumer offset management

• SQL analytics

• Docker-based local infrastructure

• Automated testing

9. Production improvements

1. Add a schema registry and event-version compatibility checks.

2. Add retry/backoff and separate transient from permanent failures.

3. Add structured logs, metrics, alerts, and consumer-lag monitoring.

4. Add integration tests with a disposable Compose stack.

5. Use secrets management, TLS, and SASL outside local development.

6. Add CI/CD and an analytics dashboard.

10. Interview explanation

“I built a Real-Time Commerce Streaming Pipeline using Python, Redpanda, PostgreSQL, and Docker Compose. The goal was to simulate an e-commerce platform where customer activities such as product views, cart additions, checkouts, and purchases are generated as events and processed continuously. Redpanda serves as the streaming broker, Python produces and consumes events, and PostgreSQL stores validated records for analytics. I implemented event validation, idempotent inserts to prevent duplicate rows, and dead-letter handling for invalid events. I also wrote SQL analytics for purchase revenue, customer activity, and product-category performance, and added automated tests. Next, I would improve monitoring, schema evolution, retry strategies, and CI/CD.”

11. Learning priorities

Kafka/Redpanda topics and consumer groups; Python JSON processing and exception handling; PostgreSQL keys, indexes, transactions, and SQL; retry and recovery strategies; Docker, monitoring, CI/CD, and secrets management.

Repository: https://github.com/shubham05jaiswal-coder/realtime-commerce-streaming-pipeline